LESSON 18

ESP32 SUPER STARTER KIT · 2-HOUR SESSION

Remote Control

Use invisible infrared light to control your ESP32 from across the room.

Name: _____ Date: _____

TODAY YOU WILL LEARN

- How an **infrared (IR)** remote sends messages with flashes of light
- How to **decode** remote buttons into code numbers
- How to choose between many options with **switch / case**
- How to make your own remote with the IR transmitter (challenge)

SESSION PLAN

0:00	Invisible light: look through a phone camera
0:10	Wire the IR receiver
0:20	Sketch 1: **L18_ReadCodes** — map every button
0:50	Short break
0:55	Add the RGB LED
1:05	Sketch 2: **L18_RemoteLights**
1:30	Challenges
1:55	Quiz and tidy up

PARTS TO COLLECT

- ☐ ESP32 board + USB-C cable, 2 breadboards
- ☐ IR receiver module
- ☐ Remote control (with its battery)
- ☐ RGB LED
- ☐ 3 resistors, 220 Ω (red-red-brown)
- ☐ 3 female-to-male Dupont wires, jumper wires
- ☐ Challenge: IR transmitter module, 2 buttons

1. Invisible light

A TV remote has a small LED on the front, but when you press a button you can't see it light up. That's because it gives off **infrared (IR)** light, a colour just past red that human eyes can't see.

> **Try it: see the invisible**
>
> Point the remote at a phone camera (the front selfie camera often works best) and press a button. You'll see the LED flicker on the screen! Cameras can see a little infrared.

The remote doesn't just switch the light on. It **flashes it in a pattern** about 38,000 times a second, like super-fast Morse code. Each button has its own pattern. The receiver turns the pattern back into a number.

> **Big idea: a protocol is an agreed language**
>
> The remote and receiver both follow the same rules, called a **protocol**. Our remote uses the **NEC** protocol: every button press sends an **address** (which device) and a **command** (which button).

2. Wire the IR receiver

The IR receiver module has 3 pins. Look closely at the labels. They are usually **–** (or G), **+** (or V, middle) and **S** (signal).

✓	IR receiver pin	ESP32 pin
☐	**–** / G	**GND**
☐	**+** / V (middle)	**3V3**
☐	**S** / signal	**14**

Install the library

Tools → **Manage Libraries…**, search **IRremote**, and install **IRremote by shirriff, z3t0, ArminJo**.

3. Sketch 1: What does each button say?

Type this in and save it as **L18_ReadCodes**. Upload it and open the **Serial Monitor** (115200 baud). Pull the plastic tab out of the remote's battery, point it at the receiver and press some buttons.

Type this in → save it as **L18_ReadCodes** — 21 lines

```
// Lesson 18: Read remote control codes
// IR receiver signal pin on 14. Press buttons and read the codes.

#include <IRremote.hpp>

const int IR_PIN = 14;

void setup() {
  Serial.begin(115200);
  IrReceiver.begin(IR_PIN);
  Serial.println("Press a button on the remote...");
}

void loop() {
  if (IrReceiver.decode()) {
    int code = IrReceiver.decodedIRData.command;
    Serial.print("Button code: 0x");
    Serial.println(code, HEX);
    IrReceiver.resume();
  }
}
```

Fill in the code for each button. **Hold** a button down. What do you notice?

Button	Code	Button	Code	Button	Code
POWER	0x_____	1	0x_____	6	0x_____
VOL+	0x_____	2	0x_____	7	0x_____
VOL–	0x_____	3	0x_____	8	0x_____
▶‖	0x_____	4	0x_____	9	0x_____
UP	0x_____	5	0x_____	0	0x_____
DOWN	0x_____	EQ	0x_____	ST/REPT	0x_____

How it works

Code	What it means
`#include <IRremote.hpp>`	Use the IRremote library (note: `.hpp`, not `.h`)
`IrReceiver.begin(IR_PIN);`	Start listening for IR on pin 14
`if (IrReceiver.decode())`	**True** when a full message has arrived and been decoded
`IrReceiver.decodedIRData.command`	The button's command number
`Serial.println(code, HEX);`	Print the number in **hexadecimal**, like `45` instead of `69`
`0x45`	How we write a hex number in code. `0x45` = 69
`IrReceiver.resume();`	"I'm done with that one, get ready for the next"

> **Getting 0x0 or weird numbers?**
>
> Make sure the remote's battery tab is out. Bright sunlight or lamps can confuse the receiver. Point the remote straight at it, from less than 2 metres.

4. Add the RGB LED

✓	From	To
☐	ESP32 pin **25**	220 Ω resistor → RGB **red** leg
☐	ESP32 pin **26**	220 Ω resistor → RGB **green** leg
☐	ESP32 pin **27**	220 Ω resistor → RGB **blue** leg
☐	RGB **longest leg** (common –)	**–** rail

5. Sketch 2: Remote control lights

Save as **L18_RemoteLights** and type this. If your codes in the table are different from the ones below, **use your codes**.

Type this in → save it as **L18_RemoteLights** 38 lines

```
// Lesson 18: Remote control lights
// Buttons 1–5 pick a colour on the RGB LED, POWER turns it off.

#include <IRremote.hpp>

const int IR_PIN = 14;
const int RED_PIN = 25;
const int GREEN_PIN = 26;
const int BLUE_PIN = 27;

void color(int r, int g, int b) {
  analogWrite(RED_PIN, r);
  analogWrite(GREEN_PIN, g);
  analogWrite(BLUE_PIN, b);
}

void setup() {
  Serial.begin(115200);
  IrReceiver.begin(IR_PIN);
  color(0, 0, 0);
}

void loop() {
  if (IrReceiver.decode()) {
    int code = IrReceiver.decodedIRData.command;
    Serial.println(code, HEX);

    switch (code) {
      case 0x0C: color(255, 0, 0);     break;  // 1 = red
      case 0x18: color(0, 255, 0);     break;  // 2 = green
      case 0x5E: color(0, 0, 255);     break;  // 3 = blue
      case 0x08: color(255, 120, 0);   break;  // 4 = yellow
      case 0x1C: color(255, 255, 255); break;  // 5 = white
      case 0x45: color(0, 0, 0);       break;  // POWER = off
    }
    IrReceiver.resume();
  }
}
```

How it works

Code	What it means
`switch (code) { … }`	Look at `code` and jump to the matching `case`
`case 0x0C: … break;`	If `code` is `0x0C`, do this, then `break` out of the switch
(no matching case)	Nothing happens. Other buttons are ignored
`color(255, 120, 0);`	Our function from the PWM lesson: red, green, blue from 0 to 255

switch vs. lots of ifs

`switch` does the same job as `if (code == 0x0C) … else if (code == 0x18) …`, but is easier to read when there are many choices. Don't forget the `break`! Without it, the code "falls through" into the next case.

Try it

1. Add buttons 6, 7, 8 and 9 with your own colours (purple, cyan, pink…).
2. Does it work if you bounce the IR off a wall or the ceiling?
3. Try your TV remote at home (or a friend's). Do its buttons give different codes?

6. Challenges

Challenge 1: Brighter, dimmer

Make **VOL+** and **VOL–** change the brightness. *Hint: keep a* `brightness` *variable from 0 to 100, and use* `r * brightness / 100` *in* `analogWrite`.

Challenge 2: Power toggle

Make **POWER** switch the light off **and** back on to the same colour. *Hint: a* `bool isOn` *and* `isOn = !isOn;`

Challenge 3: Party mode

Button **0** starts random colours every 150 ms until another button is pressed. Use `random(256)`.

Bonus: Build your own remote

The kit has an **IR transmitter** module too. With a friend, wire it to one ESP32 (signal on pin **4**, power to 3V3, GND to GND) plus two buttons on 13 and 14. Use `IrSender.begin(4);` and `IrSender.sendNEC(0x00, 0x0C, 0);` to send "button 1" to your friend's ESP32!

7. Quiz

Check what you learned

1. Why can't you see the light from a remote control?

---

2. What does `break` do in a `switch`?

---

3. What number is `0x10` in normal (decimal) numbers? *(Hint: hex counts in sixteens)*

---

4. Name two things in your house that might use IR remotes.

Tidy up: unplug the USB cable and **put the plastic tab back in the remote** (or take out the battery) so it doesn't run flat.